PRM393 — Lab 3: From Design to Working App with Flutter and Firebase

In Lab 2 you designed a mobile app and wrote a Flutter handoff for it. In this lab you build that app in Flutter, connect it to Firebase, test it with an AI testing agent, and review your code with AI tools. The Lab 2 design is your specification: you are graded on how faithfully and how well you turn it into a working, tested, maintainable app, and on the reasoning you record along the way.

1. Lab information

Item	Detail
Course	PRM393 — Mobile Programming
Lab	Lab 3 — From Design to Working App with Flutter and Firebase
Prerequisite	Your Lab 2 submission (Figma file and `flutter-handoff.md`)
Work mode	Individual (unless the lecturer announces teams)
Duration / deadline	[to be filled by lecturer]
Weight in course grade	[to be filled by lecturer]
Required tools	Flutter and Dart, Android emulator or device, Firebase project, Provider or Riverpod, MobAI (free tier), one AI code review tool, GitHub
Platform	Android (emulator or physical device)
Submission	GitHub repository, project report, demonstration video

Tools: Firebase for Flutter · MobAI · Provider · Riverpod

2. Learning outcomes

After this lab you can:

1. Turn a design and a handoff specification into a working Flutter app, and measure how closely the result matches the design.
2. Structure a Flutter app with the MVVM pattern and Provider or Riverpod, keeping business logic out of the UI.
3. Integrate Firebase Authentication (Google Sign-In), Cloud Firestore, Cloud Messaging, Analytics, Crashlytics and Remote Config.
4. Secure user data with Firestore Security Rules and store generated files in an object storage service without exposing secrets in the app.
5. Use AI coding tools responsibly: verify what they generate, and record what you accepted, changed or rejected.
6. Automate end-to-end tests with an AI testing agent (MobAI), review the tests it writes, and judge whether a healed test is still a valid test.
7. Use AI-assisted code review to find defects and decide which findings to act on.

3. Task and scope

Build the mobile app you designed in Lab 2 as a working Flutter application for Android. The app must:

- Implement all screens of your Lab 2 final UI (at least 8 screens) with real navigation.
- Make all 3 Lab 2 user flows work end to end, including the error or recovery path.
- Use Cloud Firestore as its backend, so that data is created, read, updated and deleted for real and belongs to the signed-in user.
- Integrate the Firebase services listed in section 6.
- Follow your own design system: colors, typography, spacing and components come from the Lab 2 tokens and component library.

Design changes. A working app always reveals problems in a design. You may change the design, but every change must be recorded in `docs/design-change-log.md` with the reason, and Figma must be updated or annotated to match. Two changes are expected and do not need justification beyond a log entry: a Login screen (Google Sign-In) and a Profile screen, if your Lab 2 design does not already have them.

Out of scope. iOS builds, custom backend servers, payments, and features that were not in your Lab 2 design (except those required by this lab).

4. Required workflow

Work in this order: Read the spec, Set up, Build, Connect, Test, Review, Verify, Document. Commit after each stage.

#	Stage	Output
1	Read your Lab 2 handoff and list gaps or ambiguities	`docs/handoff-gaps.md`
2	Create the Flutter project, Firebase project and MVVM skeleton	Running empty app connected to Firebase
3	Implement tokens as `ThemeData` and components as reusable widgets	`lib/theme/`, `lib/widgets/`
4	Implement screens, ViewModels and Firestore data flow, one user flow at a time	Working flows
5	Integrate Auth, FCM, Remote Config, Crashlytics, Analytics, file export	Firebase features working
6	Write and run automated tests with MobAI	`mobai_tests/`, test report
7	Run AI-assisted code review on a pull request and act on findings	`docs/code-review.md`
8	Compare the app against the Lab 2 design	`docs/fidelity-check.md`
9	Write the report and record the video	Report, video

Keep `docs/implementation-ai-log.md` updated throughout (section 9).

5. Architecture requirements

The app must follow the MVVM (Model, View, ViewModel) pattern with either Provider or Riverpod for state management. Choose one and use it consistently.

- **Models:** plain Dart classes for your data, with conversion to and from Firestore documents.
- **Services:** the only code that talks to Firebase or an external API (for example `AuthService`, `FirestoreService`, `FileStorageService`).

- **ViewModels:** hold screen state and business logic, call services, expose loading, data and error states.
- **Views:** screens and widgets that render state and forward user actions to ViewModels. Views contain no business logic and never call Firebase directly.

Suggested structure:

```
lib/
├── main.dart
├── firebase_options.dart
├── theme/            (tokens from Lab 2 as ThemeData, spacing constants)
├── models/
├── services/
├── viewmodels/
├── screens/
├── widgets/          (component library from Lab 2)
└── utils/
```

Mapping from Lab 2. The token-to-Flutter table in your `flutter-handoff.md` becomes `lib/theme/`. The component-to-widget table becomes `lib/widgets/`. Each screen specification becomes one file in `lib/screens/` plus one ViewModel.

References

- Provider
- Riverpod
- Flutter: Use themes to share colors and font styles
- Flutter: Material components
- Flutter: Navigation and routing

6. Functional and Firebase requirements

The requirements below are the same for every topic. Apply them to the entities and screens of your own design. Call your main data type the **main entity** (for example a task, an event, an expense, a habit).

6.1 Screens and navigation

- Every screen from your Lab 2 final UI is implemented, plus a Login screen and a Profile screen if they were not in the design.

- Navigation matches your Lab 2 prototype (bottom navigation or the pattern you designed), including the behavior of Back.
- Loading, empty and error states from Lab 2 are implemented on the screens that need them and are driven by real data and real failures.

6.2 Authentication

Use Firebase Authentication with Google Sign-In. Users can sign in with a Google account, see their profile information (picture, name, email), and sign out. Unauthenticated users can only see the Login screen. On Android you must register your debug signing certificate (SHA-1) in the Firebase project for Google Sign-In to work.

6.3 Data with Cloud Firestore

- Store the main entity in Cloud Firestore, scoped to the signed-in user (for example `users/{uid}/items`). Use at least two collections or subcollections.
- Implement create, read (list and detail), update and delete for the main entity.
- Use at least one query with filtering or sorting, and at least one real-time listener so that the UI updates when data changes.
- Provide seed data or a way to create test data, so that every screen can be demonstrated.
- Write Firestore Security Rules so that a signed-in user can read and write only their own data. Keep the rules in `firebase/firestore.rules` and deploy them. Test mode rules (open to everyone) are not accepted in the final submission.

6.4 Report export and file storage

Generate a PDF report of the user's data (for example a summary and a list of items, with the same visual identity as the app). Upload the PDF to an object storage service, save its URL and metadata (file name, creation time, owner) in Firestore, and show the URL or an open action in the app. Choose one storage option:

Option	Notes
Cloudinary	Recommended free path. Use an unsigned upload preset restricted to PDF files and a maximum file size. Never put the API secret in the app.

Option	Notes
Firebase Storage	Requires the Blaze (pay-as-you-go) plan and a linked billing account since 3 February 2026. No-cost usage still applies, but set a budget alert.
AWS S3 or another service	Only with lecturer approval. The app must upload through a pre-signed URL created by a small service you control; AWS access keys must never be embedded in the app.

6.5 Push notifications (FCM)

The app receives push notifications sent from the Firebase console, both when it is open and when it is in the background. It requests the notification permission on Android 13 and above, shows received notifications in a Notification Center list (inside Profile or as a screen in your design), and opens the relevant screen when a notification is tapped. Notifications should be relevant to your topic (for example a reminder or an alert).

6.6 Remote Config

Fetch at least two Remote Config values, with in-app default values, and use them to change real app behavior (for example a limit on displayed items, a feature switch, or a banner message). Show the current values in Profile for demonstration. Evidence: change a value in the Firebase console, publish, and show the app changing.

6.7 Crashlytics

Integrate Crashlytics. Provide a demonstration control (for example in Profile) that records a handled exception and another that causes a test crash. Evidence: both appear in the Crashlytics dashboard.

6.8 Analytics

Track user activity with Firebase Analytics. Write an event plan in `docs/analytics-plan.md` (event, trigger, parameters, related Lab 2 flow) and implement it. Event names use lowercase letters, digits and underscores, and are at most 40 characters.

Event	When it is logged	Parameters (examples)
`login`	Successful sign-in	`method`
`logout`	User signs out	none
`screen_view`	Each screen is opened	`screen_name`
`view_<entity>`	Detail of the main entity is opened	`item_id`, `item_name`
`create_<entity>`, `update_<entity>`, `delete_<entity>`	The corresponding action succeeds	`item_id`
`export_pdf`	Report is exported and uploaded	`item_count`
Two or more events of your own	A key step of each of your Lab 2 flows	Chosen by you

Evidence: screenshots of the events in Analytics DebugView, included in the report.

References

- Firebase: Get started in your Flutter project
- Cloud Firestore quickstart
- Get started with Cloud Firestore Security Rules
- Firebase Cloud Messaging in Flutter
- FlutterFire: Remote Config
- FlutterFire: Crashlytics
- FlutterFire: Analytics
- Cloud Storage for Firebase: billing requirements
- Cloudinary: Client-side uploading
- Cloudinary: Upload presets
- Cloudinary: Upload and deliver PDF files

7. Technical requirements

The application must:

- Be written in Flutter and Dart and run on an Android emulator (with Google Play services, needed for Google Sign-In) or a physical Android device.
- Use asynchronous communication with Firebase and the file storage service, with loading states and clear error handling (no network, permission denied, upload failed).
- Use your own Firebase project. Each student owns one project, with Analytics enabled so that Crashlytics and DebugView work.
- Contain no secrets. The repository may include `google-services.json` and `firebase_options.dart`, which hold non-secret identifiers, but must not contain service-account keys, API secrets or AWS access keys. Access control is enforced by Security Rules, not by hiding configuration files.
- Pass `flutter analyze` without errors and be formatted with `dart format`.
- Follow the architecture in section 5 and the design system from Lab 2.

8. Design fidelity check

Compare your app with your Lab 2 design and report the result honestly in `docs/fidelity-check.md`. This is not a search for a perfect match; it is evidence that you know where the app differs from the design and why.

1. **Screen comparison.** For every screen, place the Figma screenshot next to an app screenshot taken on the same size (360 × 800 dp) and list each difference with a reason (design problem found, technical limit, or mistake not yet fixed).
2. **Token check.** Colors, text styles, spacing and corner radius come only from `lib/theme/`. Search the code for hard-coded color and size values outside the theme and list each exception with a reason.
3. **Component check.** Every component from your Lab 2 library exists as one reusable widget with the same variants and states, and screens use it instead of re-building it.
4. **State check.** Loading, empty and error states from Lab 2 appear on the app screens, shown with screenshots of real situations (for example airplane mode for the error state).
5. **Accessibility check.** Repeat your Lab 2 checklist on the running app: contrast, touch target of at least 48 × 48 dp, and layout at 360 dp and 412 dp widths.
6. **Handoff gaps.** In `docs/handoff-gaps.md`, list every place where your own `flutter-handoff.md` was unclear, incomplete or wrong, and how you resolved it. This is feedback on your Lab 2 specification and is graded as part of the process.

9. AI-assisted implementation

You may, and are encouraged to, use AI coding tools to write code. The rule from Lab 2 continues: AI writes, you verify and decide.

- Keep `docs/implementation-ai-log.md` (template in Appendix A) with at least 6 entries covering different parts of the app (for example theme, a ViewModel, Firestore rules, Firebase setup, PDF export, tests).
- Each entry records the prompt (verbatim), a summary of the output, how you verified it (ran it, tested it, read the documentation, wrote a test), and your decision: accepted, modified or rejected, with the reason.
- Code you cannot explain counts against you. You may be asked to explain any file of your repository in the review described in section 14.
- Log every AI tool you used, including tools in your editor.

10. Automated testing with MobAI

You verify your app with end-to-end tests driven by MobAI, a desktop app that lets an AI agent see and operate a phone or emulator. You describe a test in plain English, the agent explores the running app and writes a `.mob` script (readable lines such as tap, type, wait for, assert), and the script can be re-run as a regression test. The free tier is enough for this lab (one device at a time). Install MobAI, start your Android emulator, and run your app on it.

Required test scenarios. Write at least these 10 tests, one `.mob` script each:

#	Scenario	Must verify
1	Google Sign-In with a dedicated test account	The main screen is shown after sign-in
2	Lab 2 flow 1, happy path, start to end	The goal screen or confirmation of the flow is shown
3	Lab 2 flow 2, happy path, start to end	Same
4	Lab 2 flow 3, happy path, start to end	Same
5	The error or recovery path of your Lab 2 flow	The error message, and that recovery works

#	Scenario	Must verify
6	Create, edit, then delete the main entity	Each change is visible in the list and detail screens
7	Navigate to every top-level destination	Each screen shows its expected content and states
8	Profile and Remote Config	Profile data and the Remote Config values are displayed
9	PDF export	The upload succeeds and a URL or open action appears
10	Sign out	The Login screen is shown

Rules for the tests

- Read every script the agent writes and edit it. An assertion that only proves the taps happened is not a test; each test must check a visible outcome.
- Document the starting state (account, seed data) so that each test can be re-run from a clean state. Never commit account credentials.
- Use a separate Google account created for testing. Google may show extra screens during sign-in; handle them in the script, or, if a step cannot be automated reliably, keep a documented manual step and explain why.
- Store scripts in `mobai_tests/` and explain each test briefly in `docs/test-report.md`.

Two exercises that check you understand the tests

1. **Healing.** Make a small deliberate UI change (for example rename a button label), run the affected test, let the tool patch the script, and record the difference. Decide whether the patch only updated how the element is found, or whether it weakened what the test checks.
2. **Catching a real bug.** Introduce a real defect (for example make saving an item silently do nothing) and show that a test fails. Then remove the defect and show it passes. A suite that heals itself around a real bug is worse than no suite.

Evidence. The `.mob` files in the repository, screenshots or logs of a full run, a results table (pass or fail per test), the two exercises above, and a short explanation of each test.

Tool note. MobAI is a third-party product whose features and free-tier limits can change. If you cannot use it, tell the lecturer before switching to another tool.

References

- MobAI
- MobAI on GitHub
- mobai-mcp: connect Claude Code, Cursor and other AI tools to MobAI

11. AI-assisted code review

Before submission, review your code with at least one AI review tool, such as GitHub Copilot Code Review, CodeRabbit, SonarQube or Kodus AI. Work on a feature branch and open at least one pull request into `main`, so that tools that review pull requests can comment on it.

- The review must identify at least 3 issues, warnings, bugs, code smells or improvement opportunities in your own code.
- At least one finding must concern security or robustness (for example Security Rules, authentication handling, error handling, or secrets). Also review your `firestore.rules` with the tool.
- Record each finding in `docs/code-review.md` with: tool, file and line, description, severity in your own judgment, and your decision (accepted, modified or rejected) with the reason.
- Fix the findings you accepted and show the change (commit link or before and after code).
- Include screenshots of the review in the report.

A tool's suggestion is not always right. A rejected finding with a sound reason is as valuable as an accepted one.

12. Deliverables

12.1 Source code

A GitHub repository named `PRM393_Lab03_<StudentID>` containing:

```
PRM393_Lab03_<StudentID>/
├── README.md               (topic, Lab 2 Figma link, how to run, Firebase
and storage setup notes)
├── lib/                    (MVVM structure from section 5)
├── android/                (including google-services.json)
├── firebase/
|   ├── firestore.rules
|   └── storage.rules       (if Firebase Storage is used)
├── mobai_tests/            (one .mob script per test)
├── docs/
|   ├── handoff-gaps.md
|   ├── design-change-log.md
|   ├── fidelity-check.md
|   ├── analytics-plan.md
|   ├── implementation-ai-log.md
|   ├── test-report.md
|   └── code-review.md
└── assets/                 (screenshots: app, Firebase console, DebugView,
tests, review)
```

The repository must also contain the assets the app needs to run. It must not contain secrets (section 7).

12.2 Project report

A report of about 5 to 10 pages containing:

- Project overview and your Lab 2 topic, persona and flows
- System architecture and how MVVM is implemented
- Firebase integration design (Auth, Firestore data model and rules, FCM, Remote Config, Crashlytics, Analytics, file storage choice and reason)
- Screenshots of every implemented feature
- Analytics events with DebugView evidence
- Crashlytics reports and Remote Config demonstration
- Design fidelity check summary and design changes
- MobAI test scenarios, results, and the two exercises
- AI-assisted implementation summary
- AI-assisted code review findings and decisions
- Challenges encountered and lessons learned

12.3 Demonstration video

A video of about 3 to 5 minutes showing:

- Google Sign-In and sign out
- Your 3 Lab 2 flows working end to end, including the error or recovery path
- Create, edit and delete of the main entity, and data visible in the Firestore console
- PDF export, upload and the resulting URL
- A push notification received from the Firebase console
- A Remote Config value changed in the console and the app reacting
- A handled exception and a test crash appearing in Crashlytics
- Analytics events in DebugView
- A full MobAI test run
- The AI-assisted code review

13. Grading rubric (100 points)

Each sub-criterion is graded as full marks, partial marks or zero. Partial marks are given when the requirement is met in part, for example for some flows, screens or services but not all.

Criterion	Pts	Full marks when	Points are lost when
Functional requirements	**25**		
Screens and navigation	5	All Lab 2 screens (at least 8) exist; navigation and Back match the Lab 2 prototype	Missing screens; navigation differs without a design-change entry
User flows	10	All 3 flows work end to end on real data, including the error or recovery path	A flow breaks midway; error path missing; flows work only with fake data
Data operations	10	Create, read, update, delete; filtered or sorted query; real-time listener; loading, empty and error states driven by real data	Operations missing; UI does not update on change; states are only static screens

Criterion	Pts	Full marks when	Points are lost when
Firebase integration and analytics	**25**		
Authentication	4	Google Sign-In, profile shown, sign out, unauthenticated users see only Login	Sign-in works only in part; screens reachable without login
Firestore and Security Rules	6	Data is per user; deployed rules allow only the owner; not test mode	Open rules; data not separated per user; rules not in the repository
Report export and file storage	4	PDF generated, uploaded, URL and metadata saved and shown; no secrets in the app	Upload fails; secrets in code; no metadata
Push notifications	3	Received in foreground and background; Notification Center; tap opens the screen	Only one state works; no Notification Center
Remote Config	3	At least two values change real behavior; defaults set; console change shown	Values only displayed; no defaults
Crashlytics	2	Handled exception and test crash visible in the dashboard	Only one; no evidence
Analytics	3	Event plan written; all required events implemented with parameters; DebugView evidence	Events missing; no plan; no evidence
Architecture	**10**		
MVVM separation	5	Views contain no business logic and never call Firebase; services are the only Firebase access	Logic in widgets; Firebase called from screens

Criterion	Pts	Full marks when	Points are lost when
State management and structure	5	Provider or Riverpod used consistently; structure matches section 5	Mixed approaches; flat or unclear structure
Design fidelity and UI quality	**10**		
Tokens and components	4	Theme from Lab 2 tokens; one reusable widget per Lab 2 component; few justified exceptions	Hard-coded values across screens; duplicated components
Screens and states match design	3	Fidelity check shows small, explained differences; states implemented	Large unexplained differences; missing states
Accessibility	3	Contrast, 48 dp targets and width checks repeated on the running app	Checks not repeated; failures left unaddressed
Automated testing with MobAI	**15**		
Coverage	5	All 10 scenarios exist as `.mob` scripts and pass from a documented clean state	Scenarios missing; tests depend on manual setup that is not documented
Assertion quality	4	Each test checks a visible outcome; scripts were reviewed and edited by the student	Tests only tap; agent output submitted unchanged
Healing exercise	3	UI change made, patch recorded, and a sound judgment on whether the patch was legitimate	No exercise; no judgment
Catching a real bug	3	A real defect makes a test fail, and it passes again after the fix, with evidence	No failing run shown
AI-assisted code review	**5**		

Criterion	Pts	Full marks when	Points are lost when
Findings	3	At least 3 real findings on your own code, including one on security or robustness	Fewer than 3; findings are generic or not about your code
Decisions and fixes	2	Each finding accepted, modified or rejected with a reason; accepted ones fixed with evidence	No reasons; no fixes shown
AI-assisted implementation and spec feedback	**5**		
Implementation log	3	At least 6 entries with verbatim prompt, verification and decision	Prompts summarized; no verification
Handoff gaps	2	Honest list of gaps in your own Lab 2 handoff and how each was resolved	Missing or empty
Report and demonstration	**5**		
Report	3	All required parts, clear screenshots and evidence	Parts missing; evidence missing
Video	2	Shows every item in section 12.3 in the time limit	Items missing; unreadable or too long

Access rule. If the repository cannot be opened at the deadline, the whole submission is graded 0 until it is fixed, and the late policy applies.

14. Academic integrity, verification and submission

AI use is expected. Submitting code or tests you do not understand is not allowed.

- Git history must show work over time, with meaningful commit messages (at least [N] commits over at least [N] days), and at least one merged pull request.
- The Firebase console of your own project must show your real data, users and events.

- Each student may be asked for a review of about 5 to 10 minutes: run one flow, explain any file the examiner chooses (including a ViewModel, a service and the Security Rules), and make a small change live. Marks in the process criteria can be adjusted after this review.
- Undisclosed AI use, fabricated log entries or test results, and copying another student's work are handled under the university's academic integrity policy.

Submission checklist

- [ ] Lab 2 Figma link and `flutter-handoff.md` referenced in the README
- [ ] `docs/handoff-gaps.md` and `docs/design-change-log.md` written
- [ ] App runs on an Android emulator or device
- [ ] All Lab 2 screens and the 3 flows work on real Firestore data
- [ ] MVVM structure with Provider or Riverpod; no Firebase calls from screens
- [ ] Google Sign-In, sign out, and Login-only access when signed out
- [ ] Firestore Security Rules deployed and saved in `firebase/`; not test mode
- [ ] PDF exported, uploaded, URL and metadata saved; no secrets in the repository
- [ ] Push notification received in foreground and background
- [ ] Remote Config changes real behavior, shown live
- [ ] Handled exception and test crash visible in Crashlytics
- [ ] Analytics plan written and events visible in DebugView
- [ ] 10 MobAI tests in `mobai_tests/`, run report, healing exercise, bug-catching exercise
- [ ] Pull request reviewed by an AI tool; at least 3 findings recorded with decisions
- [ ] `docs/fidelity-check.md` and `docs/implementation-ai-log.md` complete
- [ ] `flutter analyze` passes
- [ ] Report and video submitted; repository opens in a private browser window